

The transformer as a compute object

FLOPs you can count on your fingers, bytes you can draw on a napkin, and the one honest number — MFU — that says whether your cluster is earning its invoice

Your first multi-GPU run is alive. The loss goes down, `nvidia-smi` says every GPU is at 100%, and the invoice says the node costs \$70 an hour. Then a colleague asks the only question that matters: *what MFU are you getting?* You compute it — the arithmetic takes one line — and the answer is 23%. Every metric you trusted was green, and the machine you are paying for peak performance is delivering less than a quarter of it: of every dollar on that invoice, seventy-seven cents are buying idle silicon. The gap between "GPUs busy" and "GPUs working" is invisible to every dashboard except the one you build from first principles, and building it needs exactly two skills: counting the FLOPs a transformer *must* perform, and counting the bytes it *must* move. Chapter 6 gave you the wholesale number — $C \approx 6ND$ — good for budgets and whiteboards. This chapter is the retail version: where those FLOPs live layer by layer, why backward costs exactly twice forward, where the bytes hide, and how the ratio of the two — arithmetic intensity — decides, operation by operation, whether your GPU is a calculator or a very expensive memory bus. By the end, "why is my MFU low?" stops being a mystery and becomes a checklist.

WHERE ONE LAYER'S FLOPs GO (our 7B: $d=4096$, $d_{ff}=11008$, $s=4096$ — per token, forward)

SwiGLU MLP — 270 MF (57%) three $d \times d_{ff}$ matmuls: gate, up, down	Q,K,V,O — 134 MF (28%) four $d \times d$ matmuls	QK^T, AV 67 MF (14%)
---	--	--

everything else — norms, RoPE, softmax, residual adds — is ~1% of FLOPs (but NOT ~1% of time: \$8.4)

weight FLOPs scale with N

2 FLOPs per parameter per token —
the $2N$ of Chapter 6, now with an address:
per layer: $2 \times (4d^2 + 3d \cdot d_{ff}) = 405 \text{ MF}$
 $\times 32$ layers + $2dV$ head = 13.2 GF/token

pairwise FLOPs scale with s — no weights

$4 \cdot s \cdot d$ per token per layer: 67 MF at $s=4K$,
134 MF at 8K, 536 MF at 32K —
the term $6ND$ ignores; +16% here at 4K,
the whole story by Chapter 13's crossover $s^* \approx 6d$

Figure 8.1 — One layer under the FLOP microscope. At training context lengths the MLP is the majority shareholder and attention's famous quadratic term is the junior partner — 14% at 4K for this shape. The split is the map for every efficiency chapter that follows: Chapter 13 attacks the orange box, Chapters 14–16 the whole bar.

8.1 One forward pass, counting FLOPs layer by layer

Everything in this chapter rests on one fact about matrix multiplication that you can verify on a napkin: multiplying an $m \times k$ matrix by a $k \times n$ matrix takes $2mkn$ floating-point operations — mkn multiplies, mkn adds, no way around either. A transformer is, to within a percent, a scheduled sequence of matmuls (Chapter 3), so counting its FLOPs is bookkeeping, not analysis. Let's do the books for the 7B configuration this book has carried since Chapter 3 — 32 layers, $d = 4096$, 32 heads, $d_{\text{ff}} = 11008$, vocabulary 32,000, $N = 6.74\text{B}$ parameters — for one token flowing through one layer, at sequence length $s = 4096$.

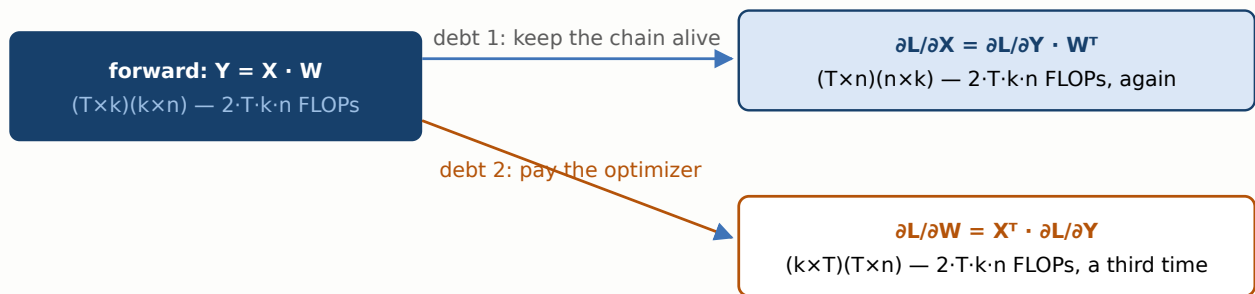
The attention block first. Projecting the token into queries, keys, and values is three $d \times d$ matmuls, and projecting the attended result back out is a fourth: $2 \cdot 4d^2 = 134$ MFLOPs. Then the part that involves no weights at all: the token's query dots against s cached keys ($2sd$ FLOPs across all heads), and the resulting scores blend s values ($2sd$ again) — $4sd = 67$ MFLOPs at $s = 4096$. The MLP is bigger than all of it: SwiGLU's three matrices (gate, up, down) cost $2 \cdot 3 d d_{\text{ff}} = 270$ MFLOPs. Per layer: 472 MFLOPs, of which 57% is MLP, 28% attention projections, 14% the pairwise term — Figure 8.1's bar. Stack 32 layers and add the output head's $2dV = 262$ MFLOPs, and one token costs **15.4 GFLOPs forward**.

Now reconcile with Chapter 6. Every matmul that touches a *weight* spends exactly 2 FLOPs per parameter per token, so the weight-bearing work is $2N = 13.5$ GF — the wholesale rule, recovered retail. The residue, 1.9 GF, is attention's pairwise term: $4sd$ per token per layer, growing linearly with context while the $2N$ term stands still. At $s = 4096$ it is a 16% surcharge on $2N$; at 32K it would be more than double the surcharge that the entire MLP stack costs, which is why Chapter 13 exists and why Chapter 6 told you to stop trusting plain $6ND$ past 8K. Two habits fall out of the exercise. First, FLOP fractions are *architecture facts, not measurements* — you can compute Figure 8.1 for any config in thirty seconds, before buying anything. Second, the things engineers spend weeks debating — norm placement, RoPE details, activation functions — are rounding errors in this ledger; the shape of the ledger is set by d , d_{ff} , s , and nothing else.

8.2 Backward: why it's twice the forward

Chapter 6 asserted the factor of three — $6N$ total against $2N$ forward — and promised the mechanism later. Here it is, and it is purely structural. Take any forward matmul $Y = XW$, where X is activations ($T \times k$) and W is a weight ($k \times n$). When the backward pass arrives at this node carrying $\partial L / \partial Y$, it owes two debts. It must hand a gradient *upstream* — $\partial L / \partial X = (\partial L / \partial Y) W^\top$, a $(T \times n)(n \times k)$ matmul — or every earlier layer starves. And it must pay the *optimizer* — $\partial L / \partial W = X^\top (\partial L / \partial Y)$, a $(k \times T)(T \times n)$ matmul — or the weight never learns. Each debt is a matmul of exactly the forward's size: same three dimensions T, k, n , reshuffled. One matmul in, two matmuls out, every weight-bearing node in the graph, no exceptions — hence backward = $2 \times$ forward, hence $6N$ per token, hence Chapter 6's entire cost model. The pairwise attention products obey the same rule (each spawns two same-shape backward products) so the surcharge triples identically.

ONE MATMUL FORWARD, TWO MATMULS BACK — THE WHOLE 6N STORY



same three dimensions each time $\rightarrow 3 \times$ forward cost per weight per token = 6 FLOPs/param/token.
 Recomputation (§8.3) runs the navy box twice $\rightarrow 8$ FLOPs — the wedge between HFU and MFU in §8.5.

Figure 8.2 — The factor of three, mechanically. Backward's two debts — a gradient upstream, a gradient for the weight — are each a full-size matmul. Note where $\partial L/\partial W$'s inputs come from: it needs the forward's X , which is why activations must be kept (or recomputed) — the bridge to §8.3.

Look once more at the orange box, because its *inputs* explain the next section. $\partial L/\partial W$ consumes X — the activation that flowed through this node *during the forward pass*, possibly seconds ago on a big model. Backward cannot run without it, so the forward's intermediate tensors are not scratch: they are IOUs that must be stored until backward redeems them, one per matmul, per token, per layer. That storage bill is the subject of §8.3, and the alternative — throw them away and rerun the forward when backward needs them — is **activation recomputation**, which converts the memory bill into a FLOP bill: the forward runs twice, cost per token rises from $6N$ toward $8N$, and your "hardware" utilization goes up while your *useful* utilization stands still. Hold that thought for §8.5, where it becomes the difference between the two utilization numbers labs report.

8.3 Bytes: the memory anatomy in depth

Chapter 4 gave you the fixed half of the bill: weights in bf16, gradients in bf16, Adam's two fp32 moments plus an fp32 master copy — **16 bytes per parameter**, so our 7B pins 108 GB of training state before processing a single token. That number is recipe-determined and batch-independent; nothing you do at runtime moves it (Chapter 15 will attack the recipe itself). The elastic half is the activations — §8.2's IOUs — and unlike the states, they scale with everything you control at launch: sequence length, microbatch size, and what you choose to keep versus recompute.

The canonical estimate is worth internalizing because it turns OOMs from surprises into arithmetic. For a standard transformer layer holding bf16 activations, storing every intermediate that backward wants costs approximately¹

$$\text{bytes per layer} \approx s b d \left(34 + 5 \frac{a s}{d} \right)$$

where s is sequence length, b microbatch size, d the hidden size, and a the head count. The 34 is the mundane part — the inputs to every matmul, norm, and nonlinearity, a fixed number of $s b d$ -sized tensors. The $5as/d$ term is the monster: the attention score and probability matrices, $a \times s \times s$ per microbatch, growing *quadratically* with context. Plug in our 7B at $s = 4096$, $b = 1$: the mundane term is 0.57 GB per layer, the attention term 2.7 GB — **104 GB across 32 layers, before the first byte of optimizer state**. This is the memory wall people actually hit, and it explains

a fact that puzzles newcomers: the 108 GB of states fits fine on two 80 GB GPUs with sharding, and then a batch size of *one* blows it up.

Two tools demolish the monster term, and you have met both. FlashAttention (Chapter 13) never materializes the $s \times s$ matrices — it recomputes attention on the fly inside SRAM tiles — deleting the $5as/d$ term entirely: our layer drops from 3.3 GB to 0.57 GB, the stack from 104 GB to 18 GB. **Selective recomputation** generalizes the idea: rank each stored tensor by bytes-saved-per-reFLOP, drop the worst offenders (norms, activations around the softmax — cheap to recompute, bulky to store), keep the matmul inputs (expensive to recompute, and backward's two debts need them). Doing this well is a solved menu, not research: the original recipe cut activation memory $5\times$ while re-spending only a few percent of the run's FLOPs, against $\sim 33\%$ for the blunt checkpoint-everything default¹. The remaining lever, microbatch size b , multiplies whatever survives — which is the real content of the advice "lower your microbatch" and the real cost of following it: smaller matmuls per call, and §8.4 is about to explain why small matmuls waste a GPU.

THE TWO HALVES OF THE MEMORY BILL (our 7B, $s=4096$ — GB, unsharded totals)

training states — fixed, 16 bytes/param

w 13.5 grad 13.5 Adam m, v + fp32 master — 81 = 108 GB whatever you do at runtime (Ch. 4; Ch. 15 attacks the recipe)

activations — elastic: s , b , and what you keep



read the chart as a launch checklist: states are Chapter 14's sharding problem; the orange bar is Chapter 13's kernel problem; the gap between the last two bars is the microbatch dial — and §8.4 prices what turning it down costs in GPU efficiency.

Figure 8.3 — Where the bytes live. Fixed states vs. elastic activations for the 7B at 4K context. The quadratic attention buffers dominate the vanilla bill and vanish under FlashAttention; after that, activation memory is a linear function of tokens in flight — which makes OOM prediction one multiplication.

8.4 Arithmetic intensity and the roofline

You now hold two ledgers for the same computation — FLOPs (§8.1–8.2) and bytes (§8.3) — and the ratio between them is the single most predictive number in GPU performance. An H100 SXM can execute 989 TFLOP/s of dense bf16 matmul but can only pull 3.35 TB/s from HBM². Divide: **~ 295 FLOPs per byte**. Any operation that performs fewer FLOPs per byte moved sits waiting on memory no matter how fast the tensor cores are; any operation above it saturates compute no matter how slow the memory is. Plot work rate against that ratio — **arithmetic intensity** — and you get the **roofline**³: a sloped ceiling where bandwidth rules, a flat ceiling where compute rules, and a ridge at 295 where they meet. Every kernel in your training step lives at some x-coordinate of Figure 8.4, fixed by its math, not by its implementation.

Place the residents. A weight matmul pushing T tokens through a $d \times d$ weight has intensity roughly $\frac{2Td^2}{2(2Td + d^2)}$ — for large T it approaches $d/2 = 2048$, a factor of seven above the ridge and gloriously compute-bound; but drive T down

and intensity falls with it, crossing the ridge near $T \approx 350$ tokens for $d = 4096$. That single calculation explains half of this book's systems advice at once: why microbatches shouldn't be tiny (§8.3's dial has a floor), why Chapter 16 packs sequences rather than padding them, why Chapter 14's pipeline stages want fat microbatches, and why tensor-parallel slicing (which shrinks each GPU's local matmul) eventually stops paying. At the other end huddle the memory-bound poor: LayerNorm, softmax, RoPE, residual adds — a handful of FLOPs per element against a mandatory read and write, intensity in the single digits, pinned to the sloped roof. They were $\sim 1\%$ of §8.1's FLOPs but consume 10–20% of wall-clock in an unfused stack, which is the entire reason kernel fusion is a genre: fusing doesn't reduce their FLOPs, it deletes round-trips to HBM. And unfused attention sits in the worst seat — its $s \times s$ intermediates force enormous traffic per FLOP — until FlashAttention lifts it over the ridge by keeping tiles in SRAM, the Chapter 13 story compressed to one x-axis jump.

One more resident matters to this book's second half: **generation**. Decoding one token per sequence re-reads all N weight bytes to spend $2N$ FLOPs per sequence — intensity $\approx$ the number of concurrent sequences. A single stream runs at intensity ~ 1 , two orders of magnitude under the ridge, which is why inference without batching uses a percent of peak, why vLLM's continuous batching exists (Chapter 25's rollout engine is an intensity-maximization machine), and why the same GPU that trains at 40% of peak can serve at 2%.

THE ROOFLINE, H100 SXM bf16 (log-log; x = FLOPs per byte of HBM traffic)

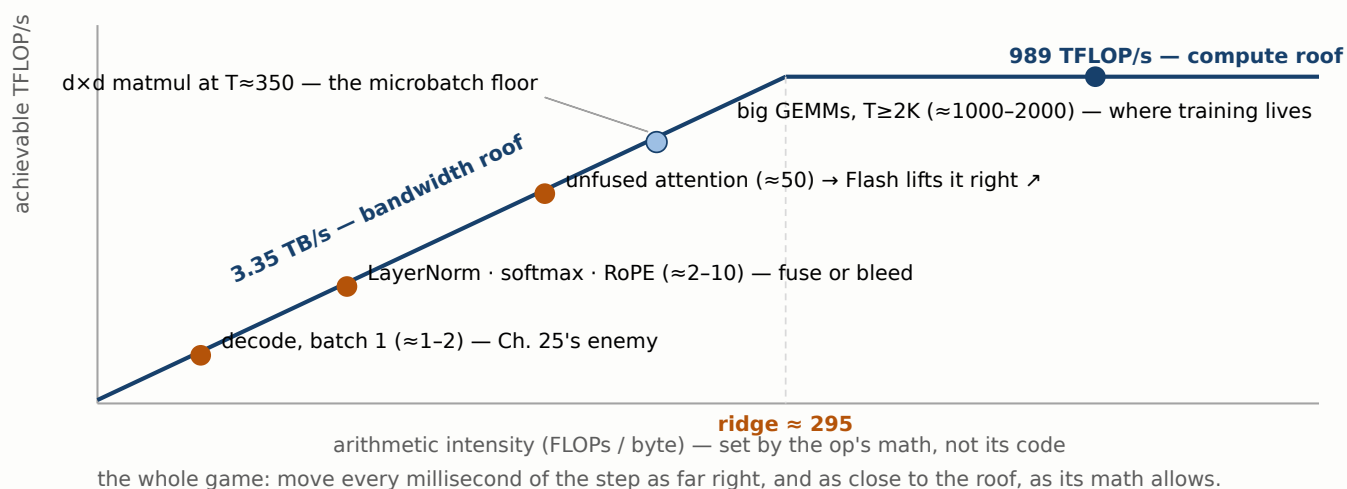


Figure 8.4 — One chart, most of the book. Ops left of the ridge are bandwidth's tenants; ops right of it can, in principle, saturate the tensor cores. Fusion moves nothing horizontally — it removes trips; FlashAttention and batching move ops rightward; shrinking local matmuls (tiny microbatches, deep TP slicing) drags them leftward toward the ridge.

8.5 MFU and HFU: how to measure them honestly

Now the number this chapter opened with. **Model FLOPs Utilization** asks the only fair question: of the FLOPs this hardware could theoretically deliver, what fraction went into the FLOPs the *model mathematically requires*? The numerator is analytic — §8.1's count, not a profiler reading — and that choice is the metric's integrity: recomputation, fancy kernels, failed and retried steps all add *work*, but they add no *model*; MFU refuses them credit. Its sibling **HFU** (hardware FLOPs utilization) counts all executed FLOPs including recomputation — useful for asking "are the tensor cores busy," dangerous for asking "am I getting what I pay for." The two bracket your recompute bill: the 530B

Megatron run reported 54.2% MFU where checkpoint-everything managed 42.1% — same hardware, same model, twelve points of invoice recovered purely by not re-spending FLOPs¹; PaLM, which coined the metric, reported 46.2% as a headline achievement of the whole systems effort⁴. The measurement is four lines and one discipline — *every input to it observed, none estimated*:

```
# MFU: the four honest lines – and the three classic ways to lie
def mfu(n_params, tokens_per_sec_global, n_gpus,
        peak_flops=989e12,                # H100 SXM, DENSE bf16. Lie #1 is 1979e12:
                                           # the "with sparsity" spec-sheet number –
                                           # 2:4 structured sparsity you are not
using.
    seq_len=None, d_model=None, n_layers=None):
    f_tok = 6 * n_params                    # Ch. 6, derived in §8.1–8.2
    if seq_len:                             # Lie #2: skipping this at long context –
        f_tok += 12 * n_layers * d_model * seq_len # fwd+bwd pairwise term
    return f_tok * tokens_per_sec_global / (n_gpus * peak_flops)

# Lie #3 lives in tokens_per_sec: count REAL tokens stepped, end to end,
# over >=30 min of steady state – dataloader stalls, checkpoint pauses and
# restarts included. Padding tokens are not tokens (Ch. 16).
mfu(6.74e9, 8 * 9800, 8)                    # -> 0.401   our 7B, one node
mfu(6.74e9, 8 * 9800, 8, 4096, 4096, 32)   # -> 0.465   with attention credit
```

The two optional arguments hide a real convention dispute: do attention's pairwise FLOPs belong in the numerator? Both conventions are published; neither is wrong; mixing them in one comparison is. Count them (they are work the model requires) when you report honestly at long context — just say so; omit them when comparing against $6ND$ -based numbers like most lab reports. The gap is 6 points for our 7B at 4K and grows with s — more than enough to fake a win over a baseline measured the other way. State your convention in the same sentence as your number; treat any reported MFU without one as ± 5 points.

8.6 What we measured on our own run

Theory earns its keep in the audit, so here is one of ours, numbers rounded but the shape exactly as it happened. A 7B-class model, one node of $8 \times H100$, $s = 4096$, bf16, the stack you would assemble from this book's parts. First measurement, day one: 5,700 real tokens per second per GPU — **23% MFU** by the code above, on a codebase whose every dashboard was green and whose GPUs read 100% busy. The audit that followed is the template we now run on every new setup, and it works because each step compares a *measured* number against an *analytic* one from this chapter, so every gap has an address.

First find: the tokenizer's length histogram showed a median document far below 4096, and the collator was padding — 27% of "tokens" stepped were padding tokens, which the honest counter refuses to count. Packing (Chapter 16's FFD pass) took measured MFU to 31% without touching a kernel: the GPUs had been doing the same work all along; the work had been worthless. Second find: a profiler trace (§8.8) showed the step's first 40 ms waiting on the dataloader — Python-side tokenization on the critical path; moving it to prefetched workers bought four points. Third: microbatch

was set to 1 "to be safe" while Figure 8.3's arithmetic said 4 fit comfortably under FlashAttention — and §8.4's ridge said b=1's matmuls were flirting with the bandwidth roof. Raising it bought most of the rest. Steady state after a week: **~9,800 tok/s/GPU, 40% MFU** (46% counting attention FLOPs), HFU a couple of points higher since we selectively recompute only norms and softmax inputs. The postmortem lesson we pass on: not one of the three finds was a GPU problem. The silicon was innocent from the start; the invoice was being spent on padding, Python, and an oversized safety margin — all visible from this chapter's two ledgers, none visible from `nvidia-smi`.

8.7 What the big labs do — reported MFU

Table 8.1 — Published utilization, with conventions attached. Read the spread as a menu of ceilings: dense models on mature stacks land in the 40s; MoE and small-expert routing pay real utilization taxes for their FLOP savings — the per-FLOP price and the per-GPU efficiency are different axes.

RUN	HARDWARE	REPORTED	WHAT TO LEARN
PaLM 540B ⁴	6,144 TPU v4	46.2% MFU	The metric's birthplace; the paper's appendix — analytic numerator, stated conventions — is still the model report card to copy
Megatron 530B ¹	A100 clusters	54.2% MFU (42.1% with full recompute)	The MFU–HFU wedge made visible: 12 points recovered by recomputing selectively instead of everything
Llama 3 405B ⁵	up to 16K H100	380–430 TFLOP/s/GPU bf16 ≈ 38–43% MFU	Reported as raw per-GPU throughput — convert yourself — and it rose mid-run with parallelism re-tuning: MFU is operated, not installed
DeepSeek-V3 671B (37B active) ⁶	2,048 H800, FP8	2.788M GPU-hours for 14.8T tokens → ≈33% bf16-equivalent MFU (our estimate from their figures)	MoE + FP8 makes "peak" ambiguous; the honest cross-lab comparator becomes GPU-hours per trillion tokens — and theirs set the record

Three patterns worth extracting. Dense models on healthy stacks cluster at 40–55%; nobody credible claims 80 — the memory-bound residue of §8.4, communication (Chapter 14), and pipeline bubbles put a practical roof in the 60s, so treat any higher claim as a convention problem until proven otherwise. MoE trades MFU for economics on purpose: DeepSeek-V3's ~33% would be a mediocre dense number, but each of those FLOPs trains a 671B-capacity model at a 37B price — per-GPU utilization and per-capability cost are different objectives, and V3 optimized the one that pays. And every lab that reports honestly reports *throughput plus conventions*, not a bare percentage — Llama 3 gives TFLOP/s per GPU and lets you do the division. Do the same in your reports: tokens per second, GPU count, peak assumed, attention convention. The percentage is derived; the throughput is the fact.

8.8 Practical considerations

Profile before you optimize, and profile with the right tool for the question. The two-ledger analysis says where FLOPs and bytes *should* go; the profiler says where the time *actually* went, and the gap between them is your to-do list. `torch.profiler` with a few-step schedule, traced to Chrome's viewer, answers the first question free: is the gap between step time and kernel time empty space (input pipeline, Python overhead, synchronization) or busy-but-slow kernels? Only the second case is a kernel problem. Nsight Systems answers the harder overlap questions (is communication hiding under compute the way Chapter 14 planned?), and one `nvidia-smi dmon` during a run tells you cheaply whether power and SM clocks even resemble their rated values — thermally throttled GPUs produce mysteriously honest-looking 15% MFU deficits. The audit order that works, from our runs and every published postmortem: input pipeline first (it's free to fix), then memory-bound fusion, then microbatch/packing geometry, then and only then parallelism surgery.

Watch MFU over time, not once. A healthy run's MFU is a flat line with a heartbeat (checkpoint dips). A sagging line means lengthening sequences (Chapter 21's ladders shift the attention surcharge mid-run), a growing straggler (one thermally-limited GPU gates the whole DP ring — Chapter 14's lesson that a cluster is its slowest member), or storage slowly failing under checkpoint load. A cliff is a topology event: NCCL falling back off NVLink, a node swap landing on a different switch. The metric doubles as ops monitoring precisely because everything upstream of the tensor cores expresses itself in it.

Common pitfalls

The sparsity peak.

Dividing by the spec sheet's "with sparsity" TFLOPs (1979 on H100) halves your reported MFU's honesty. Dense workloads compare against dense peak: 989.

Padding counted as tokens.

The single most common inflated-throughput bug. Count content tokens; padding is bandwidth spent on nothing (Ch. 16 exists to reclaim it).

Averaging over the warm-up.

The first steps compile kernels, allocate caches, and fill prefetch queues. Measure ≥ 30 minutes of steady state, checkpoint pauses included — that's what a month of training actually looks like.

Comparing MFU across attention conventions.

Six points of free "improvement" at 4K, more at 32K. State the convention or the number is noise.

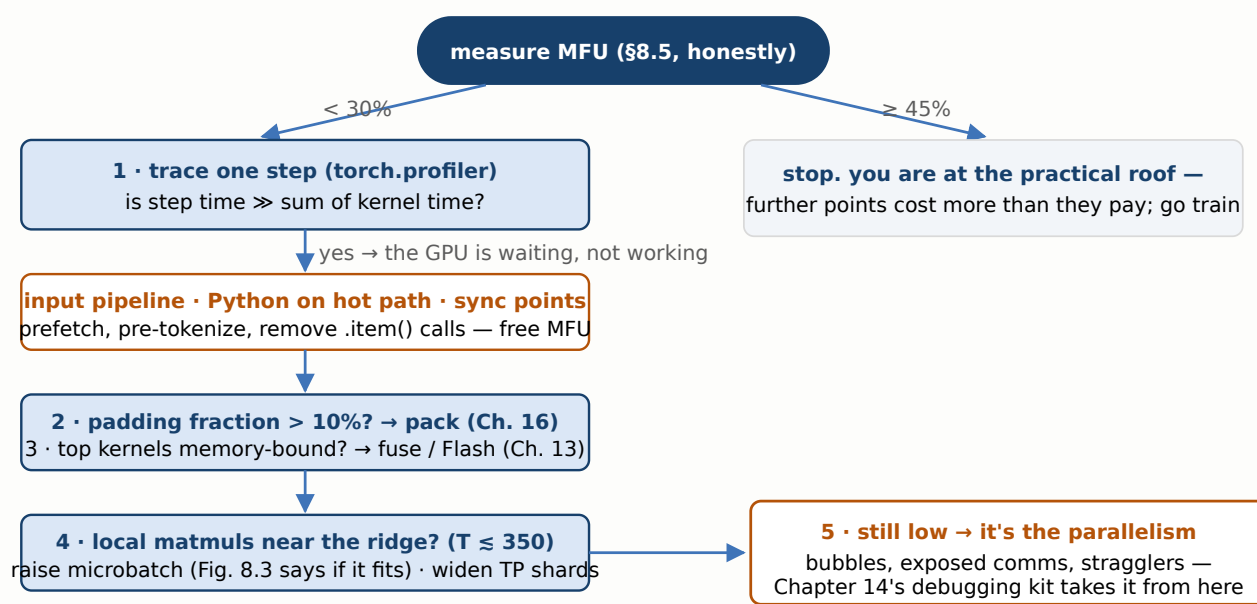
Chasing HFU.

Recompute-everything makes tensor cores gloriously busy re-deriving known values — HFU up, MFU flat, money burned. Optimize the number with the model in it.

Trusting nvidia-smi utilization.

It reports "a kernel was resident," not "the SMs were fed" — a memory-bound kernel at 3% of peak reads as 100% busy. It is a liveness check, never a performance metric.

8.9 Decision framework: finding your bottleneck



the ordering is by cost-of-fix: steps 1–2 are config changes, 3 is adopting kernels others wrote, 4 is arithmetic, 5 is engineering.

Figure 8.5 — The bottleneck hunt, ordered by effort. Most sub-30% runs die at steps 1–2 — the GPU was never the problem. The exit at $\geq 45\%$ is load-bearing: past the practical roof, a week of tuning buys two points, and the same week spent on data (Ch. 10) buys more model.

Cost-benefit, bluntly. The two ledgers of this chapter cost an afternoon to build for your config and repay it on the first OOM you predict instead of hit. The MFU counter is four lines; run it from day one — retrofitting honesty onto a month of inflated dashboards is a political problem, not a technical one. The audit sequence of §8.6 typically takes a low-30s stack to ~40% inside two weeks, mostly via packing, prefetching, and microbatch geometry — config work, worth 20–30% of your entire compute bill. Past ~45% on dense models, points get expensive: custom kernels, parallelism surgery, overlap engineering — worth it at Chapter 29's scale, where one MFU point on a 70B run is tens of thousands of dollars; not worth it on one node, where the same week of your time buys more loss reduction spent on data. **How this scales:** the ledgers don't change with cluster size — a FLOP is a FLOP on 8 GPUs or 8,000 — but the *residual* shifts: on one node your gaps are padding and Python; at Chapter 14's scale they are bubbles and network topology; at Chapter 29's they are stragglers, storage, and the failure-restart tax. The audit stays the same four lines; only the culprits graduate.

8.10 Summary

A transformer is a compute object you can fully account for with two ledgers. FLOPs: $2mkn$ per matmul compounds into 2 per parameter per token forward, 6 with backward's two structural debts (a gradient upstream, a gradient for the weight), plus attention's weightless $4sd$ -per-layer surcharge that grows with context while everything else stands still. Bytes: 16 per parameter of immovable training state, plus elastic activations — $sbd(34 + 5as/d)$ per layer — whose quadratic term FlashAttention deletes and whose remainder selective recomputation prices at a few percent of reFLOPs. The ratio of ledgers is arithmetic intensity, and one number — the ~ 295 FLOPs/byte ridge of an H100 — sorts every kernel into bandwidth's tenants (norms, softmax, small matmuls, unbatched decode) and compute's (big GEMMs), predicting from pure arithmetic why microbatches have a floor, why fusion pays, and why serving is a different economy than training. MFU is the two ledgers turned into one honest scalar — analytic model FLOPs over wall-clock peak — immune by construction to the flattery of recomputation, spec-sheet sparsity, and padding; the labs' published 40–55% dense band is your calibration, and the audit that closes the gap runs cheapest-first: padding, pipeline, fusion, geometry, parallelism. Green dashboards say the machine is busy. This chapter's arithmetic says whether it is working — and from here the book assumes you know the difference.

References

1. Korthikanti, V., et al. (2022). Reducing Activation Recomputation in Large Transformer Models. *MLSys 2023*. arXiv:2205.05198. The activation-memory formula $sbh(34+5as/h)$; selective recomputation cutting activation memory $5\times$ at a few percent FLOP overhead; 54.2% vs. 42.1% MFU on the 530B model.
2. NVIDIA (2022). NVIDIA H100 Tensor Core GPU Architecture whitepaper. Peak bf16 dense 989 TFLOP/s (1,979 with 2:4 structured sparsity); HBM3 3.35 TB/s on SXM.
3. Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: An Insightful Visual Performance Model for Multicore Architectures. *CACM* 52(4). The original roofline model this section instantiates for the H100.
4. Chowdhery, A., et al. (2022). PaLM: Scaling Language Modeling with Pathways. arXiv:2204.02311. Introduces model FLOPs utilization (MFU); 46.2% on 6,144 TPU v4; appendix B is the reference for honest FLOP accounting conventions.
5. Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. arXiv:2407.21783. 380–430 TFLOP/s per H100 in bf16 across parallelism configurations on up to 16K GPUs (≈ 38 –43% of dense peak).
6. DeepSeek-AI (2024). DeepSeek-V3 Technical Report. arXiv:2412.19437. 2.788M H800 GPU-hours for 14.8T tokens on the 671B-total/37B-active MoE with FP8 training; the $\approx 33\%$ bf16-equivalent MFU is our estimate from those published figures.